Monitoring PolyGraph

Ákos Vermes

September 3, 2026

Contents

1 Introduction **2**

2 Background **2**

3 Experimental Environment **3**

4 Methodology **3**
4.1 Separation of Event Gathering and Operational Logging 4
4.1.1 Execution-event gathering 4
4.1.2 Operational logging 5
4.2 Performance Profiling 6
4.3 Resource Usage Profiling 7
4.4 Collecting Resource Usage 7

5 Execution **8**

6 Configuration **11**
6.1 Cache configurations 11
6.2 Request Configuration 12
6.3 Warmup 13
6.4 Environment Configuration 13
6.4.1 Neo4J 13
6.4.2 ArangoDB 14

7 Results **14**
7.1 General observations 16
7.2 Obvious caching hierarchy 16
7.3 GPU timeline 16
7.4 The warm-up effect 17
7.5 Caching when ArangoDB is loaded 17

8 Conclusion **17**

1 Introduction

`PolyGraph` is a component-based Graph database that uses different storage layer databases to manipulate with data and aims to use the available hardware in the cluster where its deployed as efficiently as possible. `PolyGraph` is written in C++ and has a HTTP Server interface - implemented with Drogon and a command line interface.

The following frameworks and databases were chosen carefully to fulfill our goals that is to make some selected computation as fast as possible on a selected hardware. These pieces of software can be easily scaled by adding RAM, CPU power or GPU power to the system. Currently we are supporting the following components:

- Neo4j → Most popular graph database that is used by many enterprise applications
- ArangoDB → Graph database with focus on documents
- cuGraph (C++) → `CUDA` based library specialized for graph operations such as BFS. Currently this component can be used both as a source of truth or as a caching layer. Using it as a source of truth means that it contains nodes and relationships that could not be found anywhere else in the system. On the other hand, using it as a caching layer means that it stores nodes and relationships - more specifically the global IDs of them - that is stored by some other database - such as Neo4j - as well.
- Redis → The most popular in-memory caching layer that is added to applications
- Memory mapped region → One very popular way of boosting a system's performance is adding more RAM. For this reason, we support vertical scaling by allowing part of the graph to be loaded into RAM.

In this paper, we would like to present the currently used profiling methods for `PolyGraph` that we use on a daily basis to monitor the system with. Since we are currently implementing vertical scaling into the system, we are experimenting with different kinds of caching mechanisms. The goal of the experiments is to evaluate how much performance boost can we get from using the different caching mechanisms including `Redis`, `GPU` acceleration using the `cuGraph` library and having part of the graph loaded into a memory mapped region. While executing the benchmark, we will be profiling the whole system acceleration including GPU, CPU and memory usage.

The benchmark implementation is currently tightly coupled to the experimental environment, with several configuration parameters and hardware-specific assumptions still hard-coded. As a result, the current codebase is not yet suitable for reproducible deployment on other systems and is therefore not included as an artifact. We plan to release a generalized and documented version once the integration is complete.

2 Background

Profiling Graph databases or databases in general is a broad topic.

Designing a full Graph Database with all its functionality is work for tens of software engineers for a longer period of time. For this reason, we chose to implement only a smaller part but make it as efficient as possible. Many databases of different kinds have support for executing queries applying CRUD operations on a given dataset and these are not specific for graph databases at all. For this reason, we chose to implement some typical graph algorithms and make them as efficient as possible. The first choice is the `BFS` algorithm as a very strong basis for many different applications. As `BFS` is an algorithm that just traverses the graph in a given order, it actually can be used to implement many graph algorithms. All this was the reason why we started to implement the `BFS` algorithm and make it as fast as possible on the available hardware.

Selecting a benchmarking environment for `PolyGraph`, we decided to stick to standards. One

of most famous benchmarks for graph data is the Graph Data Council. They define multiple datasets to test the queries and algorithms on. We decided to implement support for the Social Network Benchmark - SNB.

The `SNB` dataset comes in different sizes. Scaling factors are used for scaling up or down the dataset. We are measuring different scaling factors using different hardware configuration.

Table 1: LDBC SNB dataset sizes for different scaling factors.

Scaling Factor	Node Count	Relationship Count
0.1	327,588	1,477,965
1.0	3,181,724	17,256,038
30.0	88,789,833	540,915,383

3 Experimental Environment

All experiments were performed on a `Lenovo Legion 5 Pro 16ACH6H` laptop. The system has an `AMD Ryzen 7 5800H` CPU containing 8 physical cores and 16 hardware threads, with a maximum boost frequency of approximately 4.46 GHz. The processor has 256 KiB of L1 data cache, 256 KiB of L1 instruction cache, 4 MiB of L2 cache, and 16 MiB of shared L3 cache.

The system has 32 GB of DDR4-3200 memory. GPU-accelerated experiments were executed on an `NVIDIA GeForce RTX 3070 Laptop GPU` with 8 GB of dedicated video memory and a maximum power limit of 115 W.

The experiments were executed on `Ubuntu 22.04.5 LTS with Linux kernel 6.8.0-136-generic`. Table 2 summarizes the relevant hardware configuration

Table 2: Hardware configuration of the benchmark system.

Component	Specification
System	Lenovo Legion 5 Pro 16ACH6H
CPU	AMD Ryzen 7 5800H
CPU cores	8 cores / 16 threads
CPU frequency	up to 4.46 GHz
L3 cache	16 MiB
Memory	32 GB DDR4-3200
GPU	NVIDIA GeForce RTX 3070 Laptop GPU
GPU memory	8 GiB
GPU power limit	115 W
Storage	1 TB SK hynix NVMe SSD
Operating system	Ubuntu 22.04.5 LTS
Kernel	Linux 6.8.0-136-generic

4 Methodology

There are many different aspects a database can be profiled from. We have selected two that gives a precise view of the system for the goal we have set. As usual in computer systems, to a certain degree if we keep adding hardware (either horizontally or vertically), the performance will improve. This is where the two views we have chosen for profiling meet. One of them states how fast the execution of the BFS algorithm is while the other displays the total resource usage while we were executing the algorithm.

4.1 Separation of Event Gathering and Operational Logging

PolyGraph distinguishes between *execution events* used for measurement and profiling, and *operational log messages* intended for developers and operators.

The design exposes two independent interfaces. Structured execution instrumentation is provided by the `events` subsystem, while ordinary human-readable messages are provided by the `logging` subsystem. Both may internally use `spdlog`-backed sinks, but they do not share their public API, configuration, or lifecycle.

4.1.1 Execution-event gathering

Execution events describe measurable points or intervals in the execution of an algorithm. A point event can attach structured fields to a stable event name:

```
#include "events/Events.hpp"

EVENT_INFO("bfs.frontier.expanded",
           {"depth", depth},
           {"frontier_size", frontier.size()},
           {"neighbors", neighbors.size()});
```

Timed intervals use an RAII scope. The scope emits a `<name>.start` event when it is created and a `<name>.end` event when it leaves scope. Additional fields can be attached to the end event:

```
EVENT_SCOPE_INFO(expand_span,
                 "bfs.frontier.expand",
                 {"depth", depth},
                 {"frontier_size", frontier.size()});

const auto rows = store.get_minimal_neighbors(frontier, depth);

EVENT_SCOPE_INFO_FIELD(expand_span, "rows", rows.size());
```

At runtime, enabled events are forwarded to an `EventCollector`. The collector applies the configured severity and prefix filters, adds common metadata such as the sequence number, timestamp, elapsed time, service name and thread ID, and forwards the resulting event to one or more event sinks. In the benchmark configuration, the principal sink serializes the event as JSON and writes it to `logs/polygraph.events.jsonl`. These JSONL records are the input to the existing event-log reconstruction and timeline tooling.

Event gathering also has a build-time switch:

```
cmake -S . -B build/release \
  -DPOLYGRAPH_ENABLE_EVENT_GATHERING=OFF

cmake --build build/release -j
```

When `POLYGRAPH_ENABLE_EVENT_GATHERING=OFF`, every `EVENT_*` and `EVENT_SCOPE_*` macro expands to `((void)0)`. Consequently, event names, JSON field construction, field expressions, collector lookups, timers and scope objects are not present at the instrumented call site. Event implementation source files are also omitted from the observability target. Thus a performance-sensitive statement such as

```
EVENT_INFO("bfs.layer.completed",
           {"expensive_field", compute_expensive_statistic()});
```

does not evaluate `compute_expensive_statistic()` in a build where event gathering is disabled. This is stronger than a runtime `if (events_enabled)` check because the instrumentation is removed during preprocessing and cannot contribute runtime work to the measured algorithm.

When event gathering is compiled in, individual events may still be controlled at runtime through the `events` section of the observability configuration. For example:

```
"events": {
  "enabled": true,
  "minimum_level": "debug",
  "periodic_flush_ms": 1000,
  "selection": {
    "include_prefixes": ["bfs.", "neo4j.", "arango."]
  },
  "sinks": [{
    "name": "events-json",
    "type": "rotating_file",
    "format": "json",
    "path": "logs/polygraph.events.jsonl"
  }]
}
```

4.1.2 Operational logging

Operational logging is independent from execution-event gathering. It is intended for diagnostic statements that a developer or operator should read, rather than for benchmark reconstruction. The interface uses format-style messages:

```
#include "logging/Log.hpp"

LOG_INFO("Starting BFS from {} with max depth {}", source_uuid, max_depth);
LOG_DEBUG("Selected backend {} for {} frontier vertices",
          backend_name, frontier.size());
LOG_WARN("Backend {} returned an empty batch", backend_name);
LOG_ERROR("BFS request failed: {}", error.what());
```

These messages do not create structured benchmark events and do not appear in the event timeline. They are routed through the independently configured application logger. The default configuration writes informational messages to the console and writes debug-and-higher messages to a rotating file at `logs/polygraph.log`:

```
"logging": {
  "enabled": true,
  "minimum_level": "debug",
  "periodic_flush_ms": 1000,
  "sinks": [
    {
      "name": "console",
      "type": "stdout",
      "minimum_level": "info"
    },
```

```
    {
      "name": "application-log",
      "type": "rotating_file",
      "minimum_level": "debug",
      "path": "logs/polygraph.log"
    }
  ]
}
```

The distinction is visible at call sites. For example, the BFS executable records one machine-readable lifecycle event and one human-readable log message for the same operation:

```
LOG_INFO("BFS completed with {} discovered vertices",
         result.distance.size());

EVENT_INFO("bfs.cli.completed",
           {"vertices", result.distance.size()});
```

The first statement is operational output. The second statement belongs to the measurement data model. Disabling event gathering at compile time removes only the second statement; ordinary application logging remains available.

4.2 Performance Profiling

In performance profiling, we were looking for a method to see how long do the given part of the algorithm take. This gives a starting point to help improving the performance of the system. In this view, we are measuring specific events in our implementation of the database meaning that the measuring process is part of the `PolyGraph` database's C++ source code and does not measure the used storage layer databases execution time on the database level. This means that sending a query to `Neo4j` and receiving a response can be measured but not on a deeper level.

One important feature of performance profiling is that it should be easily reproducible. This is achieved by scripts under the `tools` directory configured by benchmark configurations written in `json`.

This part of profiling is implemented using a logging mechanism that allows turning off event logging leaving 0 overhead with modern C++ macros and techniques.

The flow of the profiling works in the following way:

1. A graph algorithm, such as BFS, is executed
2. During execution, predefined instrumentation events are triggered at important stages of the query processing pipeline. These events represent operations such as query execution, backend access, data transfer, synchronization, GPU processing, or other measured phases.
3. Each event records relevant metadata, for example its type, start and end timestamps, duration, thread identifier, backend, or other operation-specific attributes.
4. PolyGraph uses spdlog as the logging infrastructure for emitting these structured events.
5. The events are serialized into JSON objects and written to an event log in the JSON Lines (`.jsonl`) format, where each line represents one recorded event.
6. The resulting JSONL log therefore contains a chronological trace of the operations performed during the BFS execution.
7. After the benchmark finishes, the event log is processed by the measurement and visualization tooling. Individual start/end events are reconstructed into measured execution intervals where necessary.

8. The reconstructed events are then used to analyze the execution, including operation durations, concurrency between threads, backend activity, GPU execution, and the contribution of individual components to the total query latency.
9. Finally, the processed event data can be visualized as timelines and aggregated statistics, making it possible to identify bottlenecks and understand how a PolyGraph query is distributed across its components. For the event-log visualization/reporting, we're using Plotly.js for rendering the interactive timelines/charts in the generated HTML reports.

4.3 Resource Usage Profiling

In this second view, we are profiling how many resources does the system as a whole use. Achieving this goal is done by the combination of standard metrics in database - and web-based systems.

1. PolyGraph exposes runtime metrics describing the behavior and resource consumption of the system during query execution.
2. Container-level metrics are collected using cAdvisor. It provides information about the resource usage of the individual Docker containers, including CPU utilization, memory consumption, network activity, and filesystem usage. The collected metrics are exposed through a Prometheus-compatible endpoint.
3. In addition to the container-level metrics provided by `cAdvisor`, PolyGraph exposes application-specific metrics through its own Prometheus-compatible metric endpoint. These metrics provide information that cannot be observed purely from the container level.
4. Prometheus periodically scrapes the metric endpoints exposed by PolyGraph and `cAdvisor`. The collected measurements are stored as time-series data and can subsequently be queried using PromQL.
5. Application and service logs are collected using Grafana Alloy. `Grafana Alloy` reads the configured log sources, attaches appropriate labels and metadata, and forwards the resulting log streams for centralized storage.
6. The collected logs are stored in Grafana Loki. `Loki` organizes logs into streams identified by labels, allowing logs originating from different PolyGraph components and containers to be efficiently queried.
7. Grafana provides the visualization and exploration layer of the observability stack. `Prometheus` is used as the metric data source, while `Loki` provides access to the collected logs.
8. Grafana dashboards visualize the behavior of PolyGraph during experiments, including CPU and memory utilization, container resource consumption, request latency, throughput, and application-specific measurements.
9. The collected metrics and logs provide a system-level view of PolyGraph and complement the fine-grained event instrumentation used to analyze individual graph algorithm executions.

4.4 Collecting Resource Usage

Resource usage metrics are collected directly inside `PolyGraph` through a small C++ metrics subsystem. The subsystem is accessed through the `MetricsAccess` interface, which provides access to the currently installed metrics service without requiring individual PolyGraph components to know how the metrics backend is implemented.

A simplified usage looks as follows:

```
auto metrics = observability::metrics();

if (metrics) {
    metrics->collect();
}
```

The actual measurements are separated into individual collectors. Each collector is responsible only for one group of related measurements, for example process statistics, memory allocation statistics, or GPU statistics. Conceptually, collectors implement a small common interface:

```
class MetricsCollector {
public:
    virtual ~MetricsCollector() = default;
    virtual void collect() = 0;
};
```

The `MetricsService` owns the configured collectors and periodically invokes them. This keeps metric collection separated from the rest of the query execution code and also makes individual collectors easy to enable, disable, or extend.

For example, the process collector reads CPU and memory information exposed by Linux, while the GPU collector obtains utilization and memory statistics from the NVIDIA runtime. Memory allocation statistics are additionally collected from jemalloc. PolyGraph uses `jemalloc` because it is well suited for highly concurrent applications such as database systems, where many threads allocate memory simultaneously. It also exposes detailed allocator statistics, which makes it possible to observe values such as allocated, active, resident, and mapped memory directly from inside PolyGraph.

The values produced by the collectors are registered in the metrics service and exposed through the PolyGraph metrics endpoint, from where they can be scraped by Prometheus and later visualized in Grafana.

5 Execution

The full initialization of PolyGraph with the `LDBC SNB` dataset requires several steps. The complete process starts with the original dataset, converts it into the PolyGraph representation, loads the persistent storage layer, prepares the configured caches and finally starts the workload against the HTTP interface.

- Download the dataset from the official website. The system scripts are capable of handling the `CsvBasic & LongDateFormatter` format. The recommended scaling factors for local execution are `0.1` and `1`.
- Decompress the downloaded dataset. For example, the scale factor `1.0` dataset can be extracted using:

```
tar --use-compress-program=unzstd \
  -xvf ~/Downloads/social_network-sf1-CsvBasic-LongDateFormatter.tar.zst \
  -C data/ldbc-snb/sf1_0/raw
```

- Prepare the dataset. This step converts the original `LDBC` representation into a PolyGraph-specific representation.

```
python3 tools/polygraph.py prepare ldbc \
  --input data/ldbc-snb/sf1_0/raw \
  --sf 1.0 \
  --domain-id ldbc-snb-sf1_0-schema-based \
  --output-dir data/ldbc-snb/sf1_0/schema-based \
  --placement-config data/ldbc-snb/placement.config.json \
  --cache-config config/cache/ldbc-cache.example.json \
  --root-count 100
```

Two important configuration files are used during this step. The placement configuration determines how the logical graph is distributed between the persistent storage layer databases. For example, different node and relationship types can be assigned to `Neo4j` and `ArangoDB`.
The cache configuration describes the additional resources available to PolyGraph. It determines how much graph data can be placed into the different cache layers, such as GPU memory, Redis and memory mapped storage. Based on this configuration, the preparation script creates the corresponding cache plan and the files required by the individual cache implementations.
The preparation step also selects a number of starting vertices using the `--root-count` parameter. These vertices can later be used by the benchmark workload, allowing the same set of starting points to be reused between different configurations.

- Load the persistent storage layer. Currently, PolyGraph uses local instances of `Neo4j` and `ArangoDB` running in containers. Their configuration is available in `storage_layer/docker-compose.yml`. For local testing, the required credentials can be configured in `storage_layer/.env`:

```
cat >> storage_layer/.env <<'EOF'

NEO4J_USERNAME=neo4j
NEO4J_PASSWORD=password
ARANGO_ROOT_PASSWORD=rootpassword
EOF
```

The prepared dataset can then be loaded using:

```
python3 tools/polygraph.py instance sf1_0 schema-based load
```

The command creates the required database instances and loads the parts of the graph assigned to them by the placement configuration. The database instances contain the scaling factor in their names, which makes it possible to keep multiple versions of the `LDBC SNB` dataset available at the same time.
During loading, PolyGraph also creates `proxy` nodes for relationships crossing storage layer boundaries. Such a relationship can, for example, have its source node stored in `Neo4j` while its destination node is stored in `ArangoDB`. The proxy nodes allow the traversal to continue between these independently stored parts of the logical graph.

- Prepare the cache layer. The cache plan generated during dataset preparation describes which parts of the graph should additionally be available through the configured cache backends.
Redis requires an explicit loading step. The generated Redis cache data can be loaded using:

```
python3 tools/polygraph.py cache redis-load \
  data/ldbc-snb/sf1_0/schema-based/domain.json \
  --reset \
  --batch-size 10000
```

The loader reads the generated cache plan and inserts the prepared adjacency data into the Redis instance. The `--reset` option removes data belonging to a previous cache before loading the newly generated one.
Not every cache backend requires a separate loading command. Memory mapped cache files are generated during the preparation phase and can be opened directly by PolyGraph. GPU graph representations are created when the corresponding cache is initialized by the running system. Therefore, the same cache configuration is used throughout the whole

process: first to generate the cache plan and later to determine which cache representations are made available during execution.

After the storage and cache layers have been prepared, PolyGraph can be started with the generated `domain.json` and the corresponding cache plan. The `domain.json` represents the logical graph together with the information needed to access its physical storage. The cache plan adds information about graph data that is also available through Redis, memory mapped storage or the GPU.

The workload is executed through the PolyGraph HTTP interface. The HTTP server is used as the common entry point of the system, meaning that the benchmark does not have to directly communicate with the individual storage and cache backends. It sends the configured graph operations to PolyGraph, while PolyGraph performs the operation using the currently active placement and cache configuration.

The HTTP benchmark itself is described by a request plan. Instead of defining a single hard-coded request, the configuration specifies which endpoint should be called, the HTTP method, parameters of the operation, the request payload and the time at which individual requests should be issued.

A simplified HTTP request plan can look as follows:

```
{
  "mode": "serial",

  "defaults": {
    "endpoint": "/bfs",
    "method": "POST",
    "measure": true,

    "payload": {
      "maxDepth": 2,
      "direction": "both",
      "returnPredecessors": false
    }
  },

  "requests": [
    {
      "id": "bfs-001",
      "atMs": 0,
      "startUuid": "2533281232918230"
    },
    {
      "id": "bfs-002",
      "atMs": 1000,
      "startUuid": "2533281232918231"
    },
    {
      "id": "bfs-003",
      "atMs": 2000,
      "startUuid": "2533281232918232",
      "payload": {
        "maxDepth": 3
      }
    }
  ]
}
```

The `defaults` section contains values shared between requests, while individual entries in `requests` can override them. This makes it possible to describe a larger workload without repeating the complete HTTP request for every starting vertex.

The `atMs` value determines when a request is scheduled relative to the beginning of the workload. Before starting the workload, the benchmark waits until the PolyGraph HTTP server is available through its health endpoint. After that, requests are issued according to the configured schedule.

The configuration can therefore describe both the operation itself and the way in which it is submitted. Parameters such as the starting UUID, maximum depth or traversal direction can be changed between requests while still using the same PolyGraph instance and the same physical graph configuration.

For every measured request, the benchmark records the execution time together with information identifying the request and the currently tested configuration. The request plan can be executed repeatedly, allowing the same workload to be run against different placement and cache configurations. The important property is that the workload remains unchanged: only the PolyGraph configuration describing the available storage and cache resources is changed.

To execute a simple benchmark, run the following command:

```
python3 tools/polygraph.py benchmark-caches \
  ldbc-snb-sf1_0 \
  --transport http \
  --output-dir measurements/sf1-0_benchmarks/cache-comparison \
  --cache-configs config/cache/benchmark/sf1/*.json \
  --http-request-plan \
    config/benchmarks/sf1-cache-http-requests.example.json \
  --cache-root \
    data/ldbc-snb/sf1_0/schema-based/caches/ \
  --planning-db \
    data/ldbc-snb/sf1_0/schema-based/cache-plan.sqlite3
```

The complete preparation workflow is illustrated in Figure 1.

6 Configuration

6.1 Cache configurations

As stated in the description as the goal of the benchmark, we are evaluating different caching strategies. In this section, we describe what strategies and how are we using it.

In each configuration, we are caching a given part of the graph (selected with cache-selection policy when building the cache) in 3 different ways: GPU, `Redis` and loaded into a memory mapped region. These three caches are configured to store a given percentage of the nodes (and `all` neighbors of those nodes).

We are testing the following configurations:

1. `01-gpu-100` → In this configuration, the whole graph is loaded to the GPU. Testing pure GPU performance (with the additional device communication) is a way of seeing what performance boost can we expect if we use GPU as a vertical layer above other storage-layer databases.

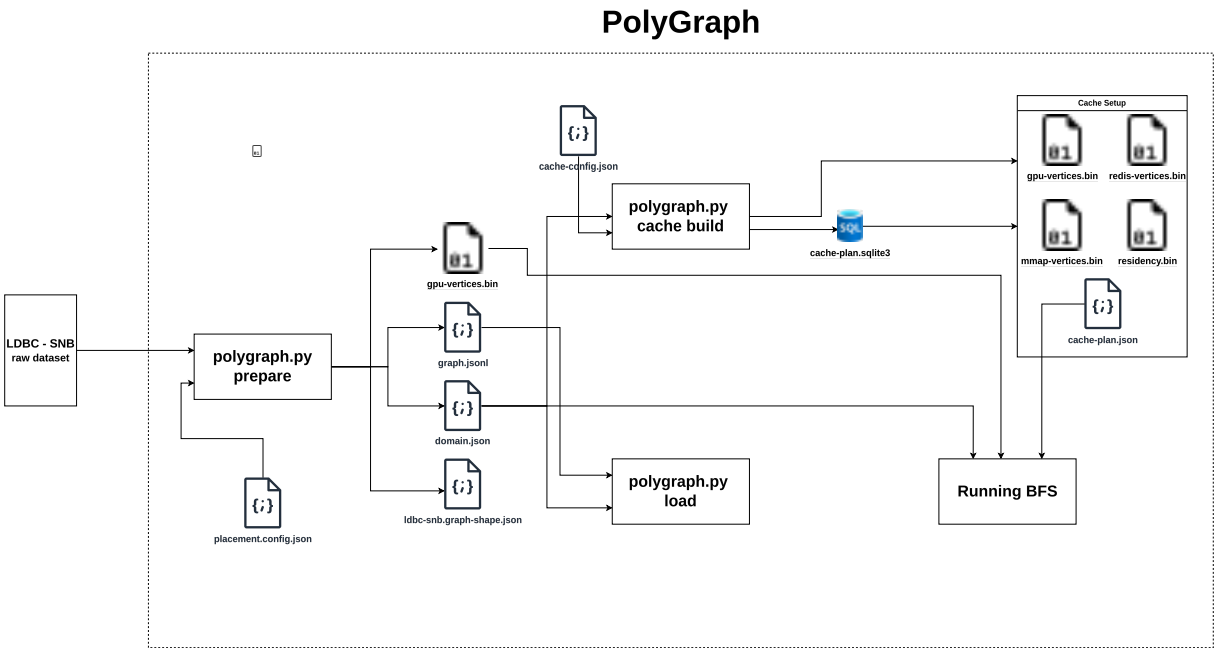


Figure 1: LDBC SNB preparation and execution workflow in PolyGraph.

2. `02-redis-100` → In this configuration, the whole graph is loaded to the `Redis`. Testing pure `Redis` performance is a way of seeing what performance boost can we expect if we use `Redis` as a vertical layer above other storage-layer databases.
3. `03-mmap-100` → In this configuration, the whole graph is loaded to a memory mapped region. Testing pure mmap performance is a way of seeing what performance boost can we expect if we use memory mapping as a vertical layer above other storage-layer databases.
4. `04-gpu30-redis20-mmap20` → In this configuration, we load 30% of the vertices to GPU, 20% to `Redis` and 20% to a memory mapped region. We benchmark how the different caching strategies work together and what performance boost can they achieve having approximately 70% of the graph cached.
5. `05-gpu20-redis15-mmap15` → In this configuration, we load 30% of the vertices to GPU, 20% to `Redis` and 20% to a memory mapped region. We benchmark how the different caching strategies work together and what performance boost can they achieve having approximately 40% of the graph cached.
6. `06-redis-25` → In this configuration, we cache 25% of the graph's vertices in `Redis`. We decided to include this configuration so we could test the performance boost of a single cache source storing approximately 25% of the graph
7. `07-mmap-25` → In this configuration, we cache 25% of the graph's vertices in a memory mapped region. We decided to include this configuration so we could test the performance boost of a single cache source storing approximately 25% of the graph
8. `08-no-cache` → In this configuration, we test how the system behaves when there is no caching used. Note that in the current configuration, we are still using GPU in this case as well just not as a caching layer above other storage-layer databases but as a source of truth storing UUIDs of nodes.

6.2 Request Configuration

In this subsection, we briefly describe what request do we actually send to the `HTTP` server. The first very important setting is that we do not send any requests in parallel. This means that we wait for the previous request to finish and only after that we send the next request.

There are multiple node and relationships types in the graph, but the starting node is always of type `Person`. We made this choice because according to our pre-calculations, nodes with type `Person` have the highest connectivity and since we are still in development phase, we chose to use this attribute so the graph was expanded already in the early phases. On the other hand, we are querying multiple concrete nodes of `Person` identified with their `UUID` attribute.

This type of benchmark has a `json` configuration where we can configure the server itself and mainly the requests sent. Each request has a string `ID` so we could follow its path, a starting `UUID` and an `atMs` attribute that determines how many milliseconds after the server is healthy the request is sent.

The benchmark was run with the following requests sent:

```
"requests": [
    {"id":"warmup-domain-root","atMs":0,"startUuid":"281483566652023","measure":
    false},
    {"id":"measured-domain-root-01","atMs":30000,"startUuid":"281483566652023"},
    {"id":"measured-domain-root-02","atMs":60000,"startUuid":"281479271686965"},
    {"id":"measured-domain-para-01","atMs":90000,"startUuid":"281494304063994"},
    {"id":"measured-domain-para-02","atMs":90000,"startUuid":"281483566652023"},
    {"id":"measured-domain-para-03","atMs":90000,"startUuid":"281494304073825"},
    {"id":"measured-domain-para-04","atMs":90000,"startUuid":"281500746522737"},
    {"id":"measured-domain-root-04","atMs":120000,"startUuid":"281505041486837"},
    {"id":"measured-domain-root-05","atMs":150000,"startUuid":"281477124201747"}
  ]
```

Note that even though the name of some requests contains the word “para” and have an identical `atMs` attribute, the requests are strictly sent one after the other. It is designed this way so in the future we could run the same configuration in a parallel setting where the server has to handle parallel requests.

We ran this set of requests a total of 10 times from which the first 7 were warm-up runs for each cache configuration. Note that there are 9 requests in total but the requests are sent with 7 unique start UUIDs so as a warm-up we decided to send only one request for each of the 7 unique UUIDs. Summing up, in total we are sending 87 requests per cache configuration.

6.3 Warmup

Since we are dealing with databases, having a warm-up phase is necessary. In this case we decided to run a full instance of the requests for every configuration before measuring it. We expect in this phase for the databases to load their data into memory from disk. Especially this is the case for the `ArangoDB` instance since according to the current `Neo4j` configuration (and its default behavior), it stores the data in memory until it fits (that is true in this case).

6.4 Environment Configuration

In this subsection, I would like to talk about the configuration of the databases.

6.4.1 Neo4J

`Neo4j` is using the following environment variables to control the resources of the container it is deployed in:

The important thing about `Neo4j` is that it keeps the graph in-memory by default until it fits. In most of our cases with the scaling factor of 1 it will fit the memory and if not, the performance will be still acceptable.

Table 3: Neo4j memory configuration.

Configuration	Value
server.memory.heap.initial_size	4 GB
server.memory.heap.max_size	4 GB
server.memory.pagecache.size	2 GB
dbms.memory.transaction.total.max	3 GB
db.memory.transaction.total.max	3 GB
db.memory.transaction.max	2 GB

6.4.2 ArangoDB

ArangoDB is using the following environment variables to control the resources of the container it is deployed in:

Table 4: ArangoDB server, cache, and query memory configuration.

Category	Configuration	Value
Scheduler	server.minimal-threads	16
Scheduler	server.maximal-threads	64
Scheduler	server.io-threads	4
Scheduler	server.scheduler-queue-size	4096
Scheduler	server.low-priority-multiplier	2
Cache	rocksdb.block-cache-size	3 GB
Cache	cache.size	2 GB
Cache	rocksdb.write-buffer-size	1 GB
Cache	rocksdb.enforce-cache-limit	true
AQL	query.memory-limit	512 MB
AQL	query.global-memory-limit	2 GB
AQL	query.memory-limit-override	false
AQL	query.log-memory-threshold	256 MB
AQL	query.max-runtime	0

The important default settings of ArangoDB may include that the graph is loaded into memory by default. We did not turn this on for testing reasons because it had worse performance when it was turned on. On the other hand, in case of ArangoDB, we can say that the warm-up phase is critical because thats when the needed part of the graph is loaded from disk to memory.

7 Results

In this section, I will evaluate the results from the benchmark looking deeply into the details of the benchmark. For this, I will use the analyzer that reconstructs the event logs into a nice timeline. I attach these reconstructed HTML pages and I will always specify which document I am talking about. Please be patient opening the HTML pages since they contain a lot of data pasted into the pages, therefore consumes a lot of memory meaning that it takes time to open and go through it sometimes.

First, we take a look at the reconstruction/compare.html page that compares the performance time of the given cache configuration runs. We see the following table and boxplot visualizing the data in the table:

Table 5: BFS execution time statistics for the evaluated run configurations.

Run configuration	n	Median [ms]	IQR [ms]	Min [ms]	Max [ms]	vs. best
01-gpu-100	80	3557.378	168.288	3367.423	3889.229	1.00×
02-redis-100	80	19120.433	398.099	18564.692	20496.008	5.37×
03-mmap-100	80	4494.152	165.378	4249.771	4954.245	1.26×
04-gpu30-redis20-mmap20	80	9386.194	558.705	8641.209	10539.962	2.64×
05-gpu20-redis15-mmap15	80	12749.708	413.833	12014.591	13990.755	3.58×
06-redis-25	80	19350.734	626.444	18457.547	20865.500	5.44×
07-mmap-25	80	17794.451	455.354	17193.668	19124.754	5.00×
08-no-cache	80	853577.766	176293.805	656109.565	1086718.348	239.95×

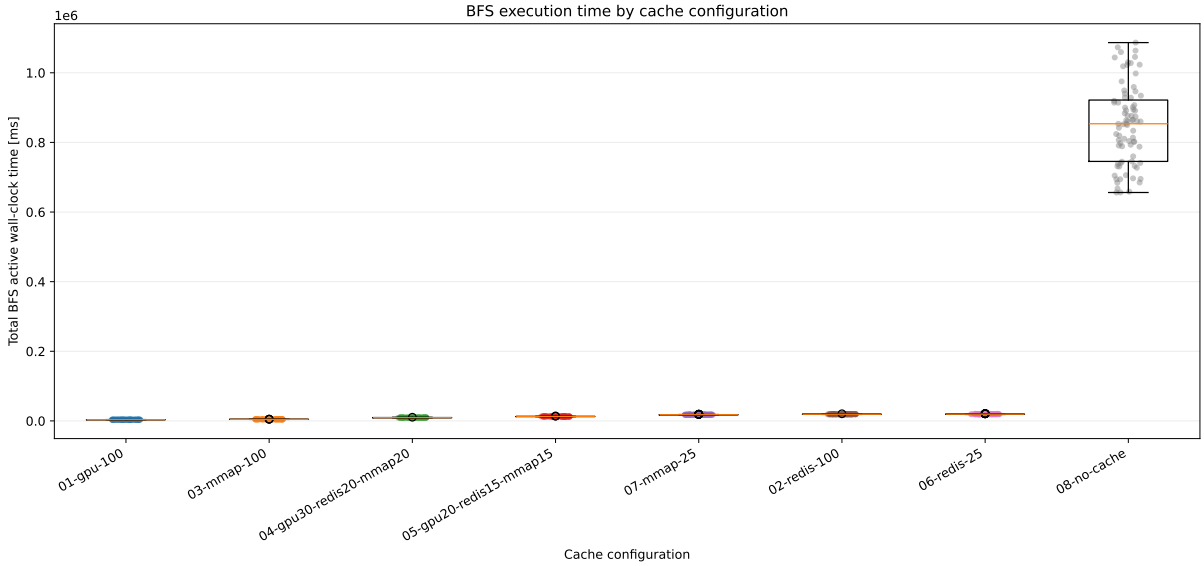


Figure 2: BFS execution time comparison across run configurations.

The measurements reported above represent the steady-state execution times. However, in several configurations, the first run exhibited a substantially slower warm-up phase before performance stabilized.

The following table includes the warm-up times:

Table 6: BFS execution time statistics for the evaluated run configurations including warm-up

Run configuration	n	Median [ms]	IQR [ms]	Min [ms]	Max [ms]	vs. best
01-gpu-100	87	3553.625	162.120	3367.423	3889.229	1.00×
02-redis-100	87	19094.043	417.775	18564.692	20496.008	5.37×
03-mmap-100	87	4502.871	175.423	4249.771	4954.245	1.27×
04-gpu30-redis20-mmap20	87	9389.947	515.827	8641.209	288619.879	2.64×
05-gpu20-redis15-mmap15	87	12768.385	436.281	12014.591	512023.230	3.59×
06-redis-25	87	19354.821	662.907	18457.547	734983.349	5.45×
07-mmap-25	87	17836.890	467.162	17193.668	697752.841	5.02×
08-no-cache	87	842109.415	171433.133	363659.213	1086718.348	236.97×

7.1 General observations

The measurements show that, after the initial warm-up phase, the cache-enabled configurations reach a relatively stable steady state. For configurations 01–07, the IQR represents only approximately 2% to 6% of the median execution time, indicating low run-to-run variability. Some experiments exhibit substantially slower execution during the first run; however, this behavior is associated with system warm-up rather than the subsequent steady-state execution. The no-cache configuration remains considerably more variable even after warm-up, with an IQR of approximately 20.7% of its median.

Just looking at the table, we can see that caching makes the performance significantly faster with the current configuration of the databases. The strongest explanation for this is that when not using any cache at all, the slice of the graph that is stored in `ArangoDB` does not fit into the memory and it keeps reloading edges evicting other useful edges from the memory that will be used in the next request. For this reason, `ArangoDB` keeps reloading the graph at every request from disk. This explains why this as the only run is not effected by the warm-up.

7.2 Obvious caching hierarchy

We can see that there is a very straightforward caching hierarchy that makes the `GPU` execution the fastest, the memory-mapped region with the `CSR` graph representation the second and the slowest the `Redis` in-memory key-value store.

Also we can notice that caching 70% of the graph and 50% of the graph helps the performance i.e. the different caches can work together and they actually boost the performance instead placing an overhead on it.

7.3 GPU timeline

According to the whole benchmark, our fastest tool to execute BFS on (a part of) the graph is using `GPU` and the `cuGraph` framework. Looking closer to the execution times of the different events at any request under `reconstruction/runs/01-gpu-100` e.g. the `requests/r004-measured-domain-para-02.ev` and its `Timeline: by thread` view, we can see that the actual "useful" execution that is looking for neighbors on `GPU` and not host-to-device or vice versa communication is a very little fraction of the execution.

The event associated with the neighbor finding on `GPU` is called `cugraph.sample.api_call`. In this concrete example, the execution of the full BFS took 3459.312 ms from which the neighbor finding events took

Table 7: Execution time by BFS depth.

Depth	Execution time [ms]
1	1.635
2	9.123
3	38.497
4	6.004
5	0.287
6	0.197

This sums up to a total of 55.743 ms. This corresponds to approximately 0.0161 of the total execution time, or 1.61%. It is a very small fraction of the full event so it would be a nice continuation to investigate whether we could improve the additional time around the execution.

7.4 The warm-up effect

As we can see, in the cases where some caching mechanism was used but we didn't cache the whole graph, there is a significant warm-up phase. This piece of information could be significant so we decided to investigate on what causes this phenomenon. Before that, the first very straightforward guess would be that one of the storage layer databases is loading the graph into memory from disk. Since we are familiar with the configuration of both databases, our first guess would be `ArangoDB`.

For this reason, we checked the event output dashboard for the `04-gpu30-redis20-mmap20` configuration. It can be found under `reconstruction/runs/04-gpu30-redis20-mmap20/dashboard.html`. Selecting the run that had a significantly slower `BFS execution call` than the others (288seconds) we open the `Timeline: by thread` view to see what the timeline of the BFS execution.

We can see that almost the full timeline is covered by two events: `arango.stream_neighbor_candidates` at depth 3 and at depth 4. This seems like a good proof to our guess that the graph is being loaded to the memory. Looking at the `Prometheus` metrics we can be sure about the correctness of the hypothesis.

Also we can see that even though we send the requests to different starting UUIDs, a single warm-up run is enough, there was no need for sending a request to every unique UUID because the very first requests loads the database and after that we do not see a significant performance boost.

7.5 Caching when ArangoDB is loaded

As described in the previous section, `ArangoDB` has a significant loading phase. However, looking at the data compares the given configuration to the best performing 100% of the graph is on the GPU configuration, we see that caching 25% of the graph in `Redis` does not have a significant difference from caching the whole graph there. This means that when `ArangoDB` has loaded the data into memory, caching the whole graph produces very similar results.

Even though this is logical since we are comparing two memory based options for graph neighbor lookup but still surprising. The two technologies use very different representation of the graph and very different communication. Also it is important to notice if we didn't use any caching, the results would be similar to the last `no-cache` configuration that is way slower than any other configuration.

8 Conclusion

The experiments presented in this paper show that the choice and placement of caching layers has a major effect on the performance of `PolyGraph`. In the evaluated configuration, keeping the complete graph on the GPU provided the best BFS execution time, followed by the memory-mapped CSR representation, while Redis was considerably slower. At the same time, the experiments with partially populated and combined caches show that the different caching mechanisms can successfully cooperate: even when only part of the graph is cached, the execution time is significantly reduced compared with the configuration without a cache.

The measurements also show that the behavior of the storage layer has an important influence on the observed performance. After the initial warm-up phase, the cache-enabled configurations reach a relatively stable steady state, with comparatively small run-to-run variation. The significantly slower first execution observed in several partial-cache configurations could be traced mainly to expensive ArangoDB accesses, where data required by the traversal is loaded from disk into memory. Once this initialization has taken place, the following executions become

substantially faster and more consistent. The no-cache configuration behaves differently and remains both significantly slower and more variable, demonstrating how strongly the current execution depends on avoiding repeated accesses to the persistent storage layer.

The fine-grained event instrumentation was also useful for identifying where the execution time is actually spent. Although GPU execution using `cuGraph` was the fastest evaluated approach, the measured calls performing the actual neighbor lookup accounted for only 55.743 ms of a 3459.312 ms BFS execution, approximately 1.61% of the total time. This indicates that further improvements are likely to come not only from accelerating the graph algorithm itself, but also from reducing the surrounding overhead, such as data preparation, communication, synchronization, and movement between the different execution layers.

Overall, the results support the basic idea behind `PolyGraph`: heterogeneous storage and execution technologies can be combined so that graph operations benefit from the strengths of the available hardware. The current benchmark also demonstrates the usefulness of combining detailed execution-event reconstruction with system-level resource monitoring, since performance differences can be connected to concrete parts of the execution rather than being observed only as end-to-end latency. These results should, however, be interpreted within the current experimental environment and database configuration. A natural continuation of this work is therefore to investigate the remaining execution overhead in greater detail, evaluate additional graph sizes and hardware configurations, and study how cache placement and execution decisions can be selected dynamically instead of being fixed before execution.